ESCUELA POLITÉCNICA NACIONAL

ISWD633 CONSTRUCCIÓN Y EVOLUCIÓN DE SOFTWARE

NOMBRE ESTUDIANTE(S):	Jorge Romario Camino Guzmán Oscar José Fierro Aules Joel Alexander Places Lucero Josué Ismael Quintana Remache
FECHA:	26-10-2026
TEMA:	Refactorización Code Smells

DESARROLLO

1. Introducción

En este trabajo se levantó SonarQube en un contenedor de Docker, se analizó con él el código de un proyecto Java, se aplicaron 15 refactorizaciones de tipos diferentes a partir del reporte obtenido y se volvió a analizar el código para medir el resultado. SonarQube es una plataforma que inspecciona el código fuente con reglas de calidad y lo califica según su seguridad, su confiabilidad y su mantenibilidad [2].

El proyecto analizado es **json-simple** [4], una librería de código abierto para leer y escribir JSON en Java. Se eligió porque es pequeño (13 archivos y 1 822 líneas de código), es fácil de entender y trae 45 pruebas automatizadas, que sirven para comprobar que cada refactorización no cambia el comportamiento del programa. Una refactorización es un cambio en la estructura interna del código que lo hace más fácil de entender y de modificar sin alterar lo que hace [1].

El trabajo se organizó en un repositorio Git con un commit por refactorización: la etiqueta `v1.0` marca el código original y la etiqueta `v1.1` el código refactorizado (sección 7). La Tabla 1 resume las herramientas utilizadas.

Tabla 1. Herramientas utilizadas

Herramienta	Versión	Uso
Docker Engine (Docker Desktop)	29.6.1	Ejecuta SonarQube y la compilación del proyecto en contenedores
SonarQube Community Build	26.10.0	Análisis estático del código
Apache Maven (imagen `maven:3-eclipse-temurin-21`)	3.10.0 · JDK 21	Compila el proyecto, ejecuta las pruebas y envía el análisis a SonarQube
JaCoCo	0.8.13	Mide la cobertura de las pruebas [7]
Git	2.55.0	Control de versiones: un commit por refactorización

2. Levantamiento de SonarQube con Docker

2.1 Creación del contenedor

SonarQube se ejecutó con la imagen oficial `sonarqube:community` de Docker Hub [3]. Antes de crear el contenedor se creó la red `net-sonar`, para que el contenedor de Maven que hace el análisis pueda llegar a SonarQube por su nombre, y tres volúmenes nombrados, para que los datos, los complementos y los registros de SonarQube no se pierdan si se elimina el contenedor [3]. La Figura 1 muestra los comandos ejecutados y su salida, y la Tabla 2 explica los parámetros de `docker run`.

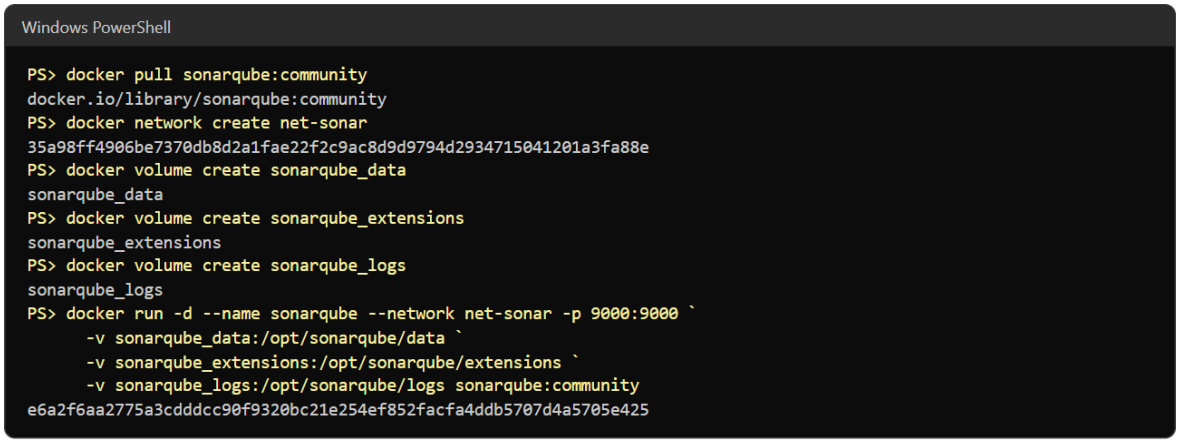


Figura 1. Creación de la red, los volúmenes y el contenedor de SonarQube

Tabla 2. Parámetros del comando docker run

Parámetro	Valor	Para qué sirve
`-d`	-	Ejecuta el contenedor en segundo plano
`--name`	`sonarqube`	Nombre del contenedor; el contenedor de Maven lo usa como dirección del servidor
`--network`	`net-sonar`	Red de tipo bridge que comparte con el contenedor de análisis
`-p`	`9000:9000`	Publica la interfaz web y la API en http://localhost:9000
`-v`	`sonarqube_data:/opt/sonarqube/data`	Base de datos embebida e índices de búsqueda
`-v`	`sonarqube_extensions:/opt/sonarqube/extensions`	Complementos (plugins)
`-v`	`sonarqube_logs:/opt/sonarqube/logs`	Registros del servidor

2.2 Verificación del servicio

SonarQube tarda alrededor de dos minutos en iniciar, porque arranca tres procesos: el servidor web, el motor de cómputo y Elasticsearch. Se comprobó que el contenedor estaba en ejecución con `docker`

`ps`, que el registro mostraba el mensaje *SonarQube is operational* y que la API respondía con el estado `UP` (Figura 2).

```
Windows PowerShell

PS> docker ps --filter name=sonarqube
CONTAINER ID   IMAGE                 STATUS          PORTS                                       NAMES
e6a2f6aa2775   sonarqube:community   Up 49 minutes   0.0.0.0:9000->9000/tcp, [::]:9000->9000/tcp   sonarqube
PS> docker logs sonarqube | Select-String "operational"
2026.10.27 01:04:50 INFO  app[][o.s.a.SchedulerImpl] SonarQube is operational
PS> curl.exe http://localhost:9000/api/system/status
{"id":"147B411E-AaF64hdXsjT478laR1ov","version":"26.10.0.133078","status":"UP"}
```

Figura 2. Verificación del contenedor y del servicio

La imagen usa una base de datos H2 embebida; por eso la interfaz muestra un aviso de que solo sirve para evaluación. Para este trabajo es suficiente; en un entorno de producción SonarQube se conectaría a una base de datos externa como PostgreSQL [2].

2.3 Primer ingreso

Al entrar a http://localhost:9000 se inició sesión con el usuario por defecto (`admin` / `admin`) y SonarQube obligó a cambiar la contraseña antes de continuar (Figura 3). Después se generó un token de análisis global (*Global Analysis Token*), que es el que usa el escáner para publicar los resultados sin enviar la contraseña del usuario.

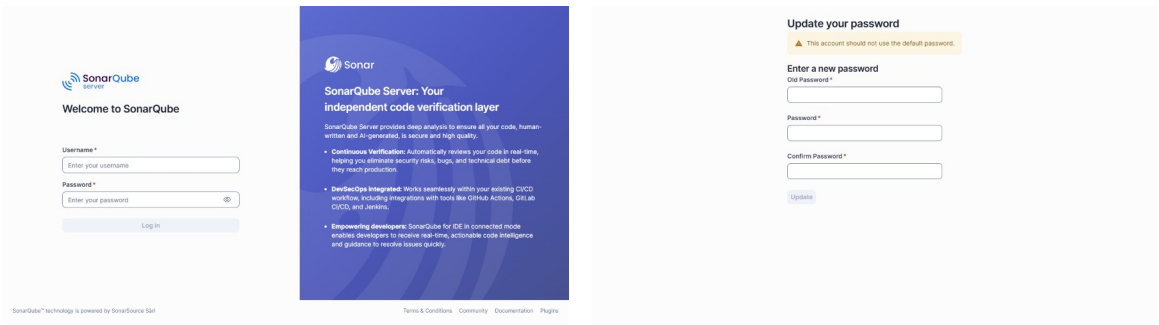


Figura 3. Inicio de sesión con el usuario por defecto y cambio obligatorio de la contraseña

2.4 Perfil de calidad

Un perfil de calidad es el conjunto de reglas que se aplica en el análisis [2]. En SonarQube 26.10 el perfil de Java por defecto es *Sonar way core*, con 274 reglas enfocadas en los problemas más críticos. Con ese perfil el análisis de json-simple no reportó ningún problema, lo que no permitía identificar qué refactorizar. Por eso se marcó como predeterminado el perfil incluido *Sonar way comprehensive*, con 618 reglas, que además revisa la legibilidad y el diseño del código (Figura 4). Los dos análisis, el de la v1.0 y el de la v1.1, usaron el mismo perfil, para que la comparación sea justa.

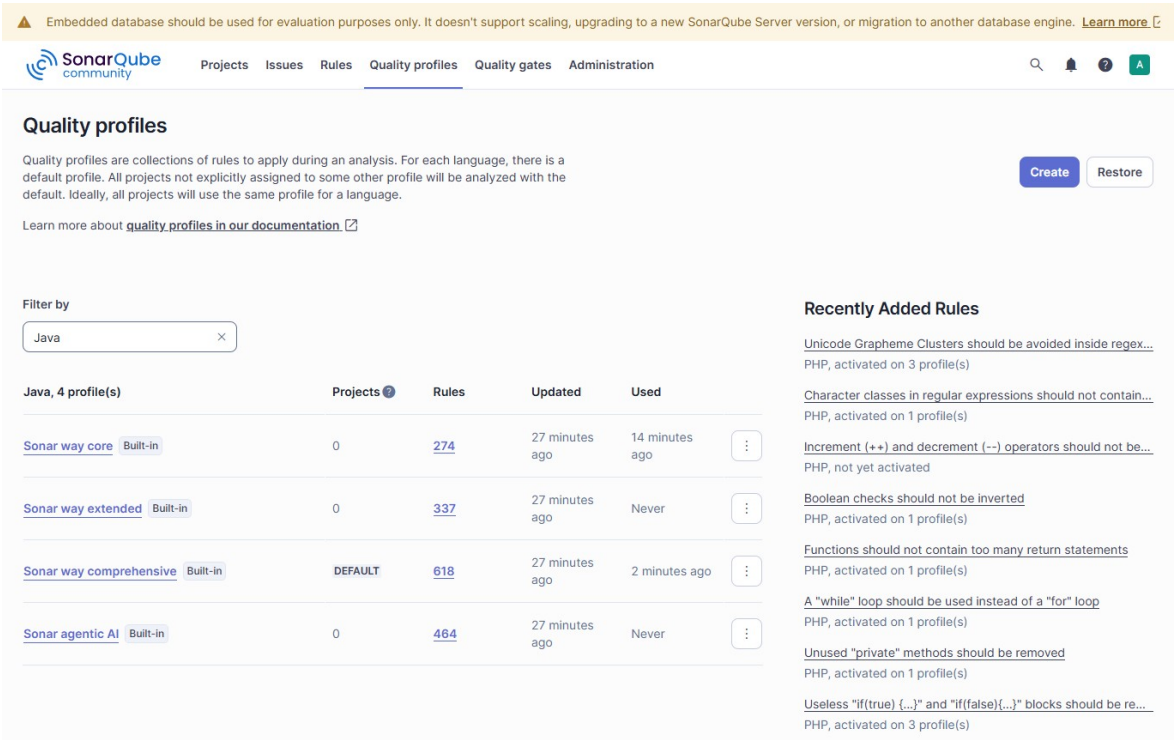


Figura 4. Perfiles de calidad de Java: Sonar way comprehensive queda como predeterminado

2.5 Ejecución del análisis

El análisis se ejecutó con el complemento de SonarQube para Maven, dentro de un contenedor `maven:3-eclipse-temurin-21` conectado a la red `net-sonar`. Así no fue necesario instalar Java ni Maven en el equipo, y el escáner encuentra al servidor en `http://sonarqube:9000`. El mismo comando compila el proyecto, ejecuta las pruebas con el agente de JaCoCo para medir la cobertura y envía los resultados a SonarQube (Figura 5). El volumen `maven-repo` guarda las dependencias descargadas entre una ejecución y otra.

```
Windows PowerShell

PS> docker run --rm --network net-sonar -v "${PWD}:/src" -v maven-repo:/root/.m2 -w /src maven:3-eclipse-temurin-21 `
      mvn -B clean org.jacoco:jacoco-maven-plugin:0.8.13:prepare-agent test org.jacoco:jacoco-maven-plugin:0.8.13:report `
      org.sonarsource.scanner.maven:sonar-maven-plugin:sonar "-Dsonar.host.url=http://sonarqube:9000" "-Dsonar.token=<token>" `
      "-Dsonar.projectKey=TG3_Equipo7-v1.0" "-Dsonar.projectName=json-simple v1.0" "-Dsonar.projectVersion=1.0"
...
[INFO] Tests run: 45, Failures: 0, Errors: 0, Skipped: 0
[INFO] Quality profile for java: Sonar way comprehensive
[INFO] Sensor JaCoCo XML Report Importer [jacoco]
[INFO] Sensor JaCoCo XML Report Importer [jacoco] (done) | time=245ms
[INFO] Sensor JaCoCo Aggregate XML Report Importer [jacoco]
[INFO] Sensor JaCoCo Aggregate XML Report Importer [jacoco] (done) | time=0ms
[INFO] ANALYSIS SUCCESSFUL, you can find the results at: http://sonarqube:9000/dashboard?id=TG3_Equipo7-v1.0
[INFO] BUILD SUCCESS
```

Figura 5. Compilación, pruebas y análisis del proyecto con Maven

3. Análisis del código v1.0

3.1 Resultados generales

La Figura 6 muestra el resumen del análisis de la versión original. El proyecto pasa el *Quality Gate* por defecto y no tiene problemas de seguridad ni de confiabilidad, pero sí 75 problemas de mantenibilidad (*code smells*). La Tabla 3 resume las métricas.

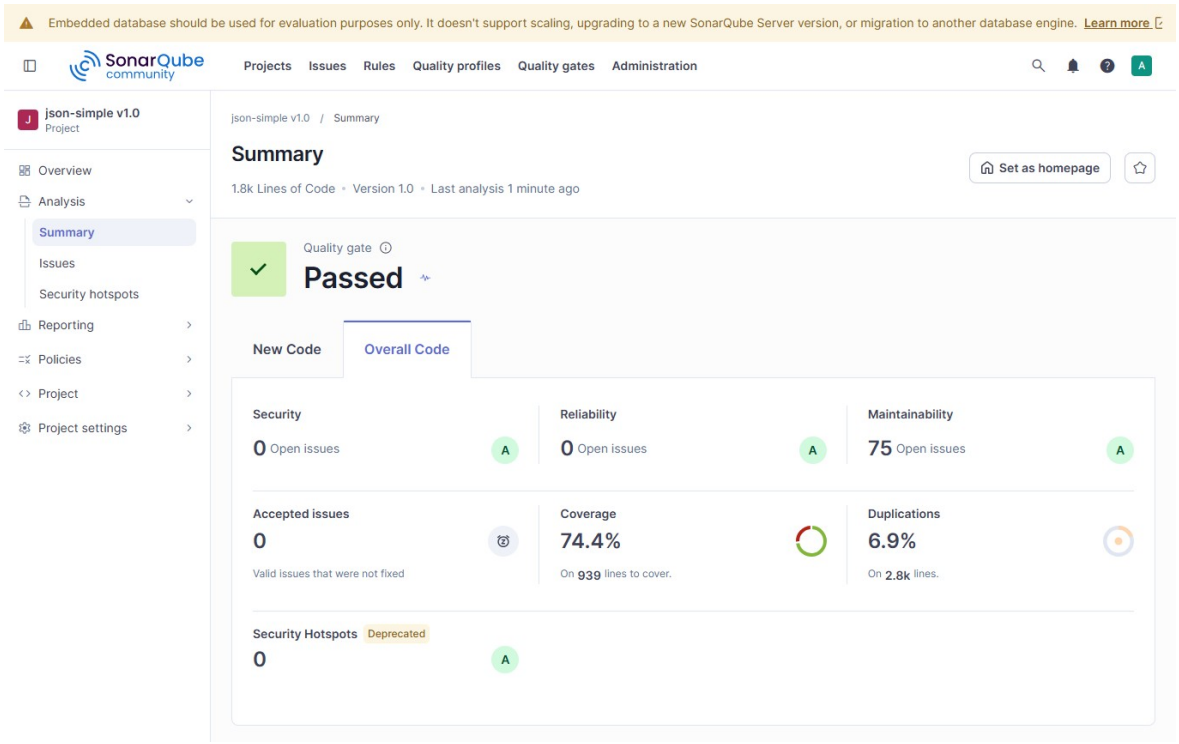


Figura 6. Resumen del análisis de la v1.0 en SonarQube

Tabla 3. Métricas de la v1.0

Métrica	Valor
Líneas de código	1 822
Archivos / funciones	13 / 127
Pruebas automatizadas	45
Bugs / vulnerabilidades / security hotspots	0 / 0 / 0
Code smells	75 (48 en el código principal y 27 en las pruebas)
Deuda técnica	872 min (≈ 14,5 h)
Calificación de mantenibilidad, confiabilidad y seguridad	A, A y A
Complejidad ciclomática / cognitiva	363 / 343
Cobertura de las pruebas	74,4 %
Líneas duplicadas	6,9 %
Quality Gate	Aprobado

3.2 Problemas detectados

De los 75 problemas, 48 están en el código principal (`src/main`) y el resto en las pruebas; el trabajo se enfocó en el código principal (Figura 7). Dentro de él, 21 problemas están en `Yylex.java`, una clase que JFlex genera automáticamente a partir de `doc/json.lex`: cualquier cambio manual se perdería al volver a generarla, por eso no se refactorizó. Los 27 problemas restantes están en código escrito a mano y corresponden a 13 reglas (Tabla 4) [6].

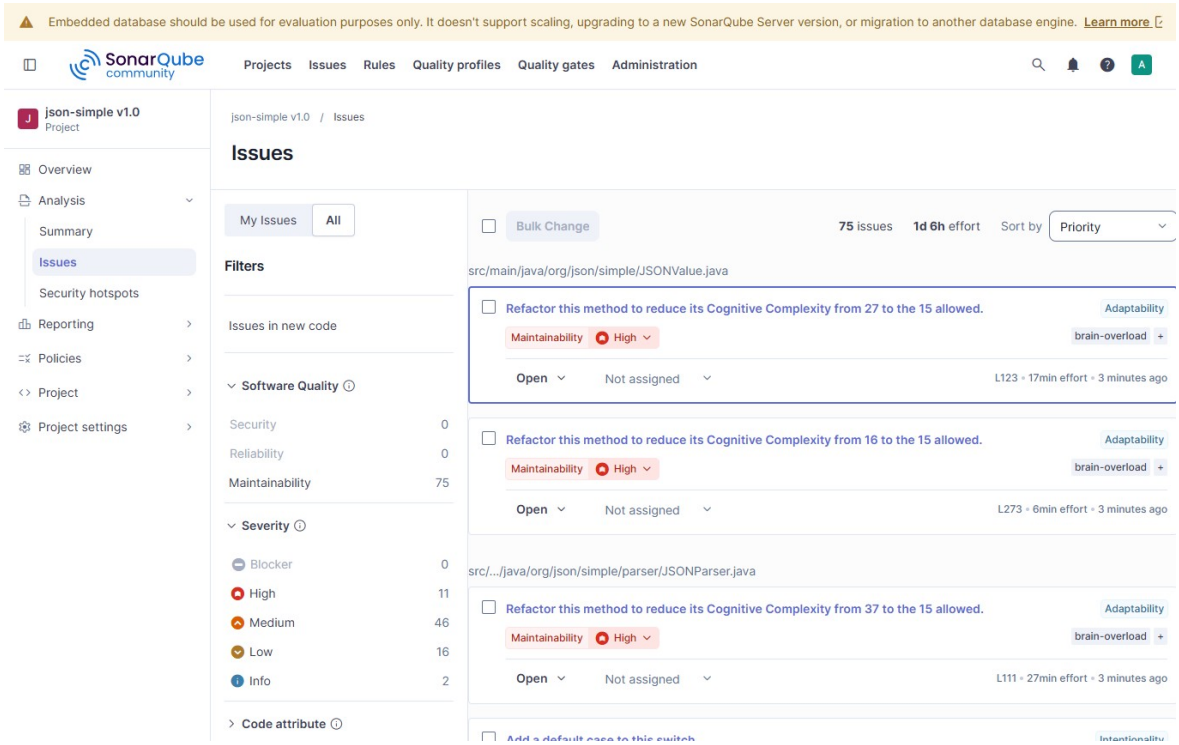


Figura 7. Lista de problemas de la v1.0, ordenada por prioridad

Tabla 4. Problemas del código escrito a mano en la v1.0

Regla	Descripción	Severidad	Cant.	Archivos	Refactorización
S1161	Usar @Override en los métodos que sobrescriben	Media	4	JSONArray, JSONObject, ParseException (2)	10
S3776	La complejidad cognitiva de un método no debe ser muy alta	Alta	4	JSONValue (2), JSONParser (2)	4, 5, 12, 13, 14, 15
S1165	Los atributos de una excepción deben ser final	Baja	3	ParseException (3)	9
S131	Los switch deben tener un caso default	Alta	3	JSONParser (2), Yytoken	6, 8
S9364	Una variable local no debe compartirse entre grupos case	Media	3	JSONParser (3)	2
S1133	El código obsoleto (deprecated) debe eliminarse	Info	2	JSONArray (2)	No (sección 5.3)
S1948	Los atributos de una clase Serializable deben ser serializables	Alta	2	ParseException, Yytoken	No (sección 5.3)
S1117	Una variable local no debe ocultar un atributo	Media	1	JSONParser	1
S1118	Una clase utilitaria no debe tener constructor público	Media	1	JSONValue	11
S1155	Usar isEmpty() para saber si una colección está vacía	Baja	1	JSONParser	No (sección 5.3)
S1181	No se debe capturar Error ni Throwable	Media	1	JSONParser	No (sección 5.3)
S1854	Las asignaciones que no se usan deben eliminarse	Media	1	JSONParser	7
S2737	Un catch debe hacer algo más que relanzar la excepción	Baja	1	JSONParser	3

El problema más grave es la complejidad cognitiva del método `JSONParser.parse(Reader, ContentHandler, boolean)`: 100, cuando el máximo permitido es 15. La complejidad cognitiva mide qué tan difícil es entender un método: suma un punto por cada `if`, `switch`, bucle o `catch`, y más puntos mientras más anidado está [5]. En la Figura 8 SonarQube marca cada uno de esos incrementos en el código.

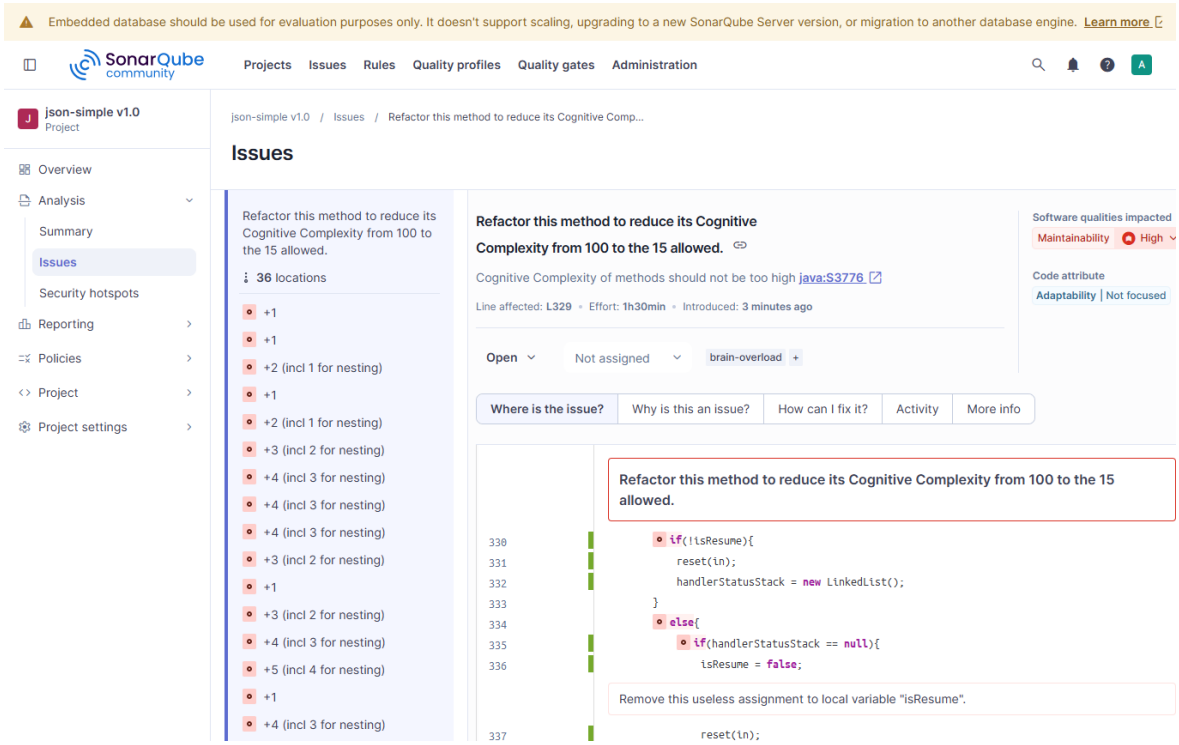


Figura 8. Detalle del problema de complejidad cognitiva de JSONParser.parse

4. Refactorizaciones

Con base en el reporte se aplicaron 15 refactorizaciones de tipos diferentes, tomadas del catálogo de Fowler [1], [8], [9], salvo la refactorización 10, que es una práctica propia de Java recomendada por SonarQube. Cada una resuelve uno o más problemas del reporte y se guardó en un commit propio (Tabla 5).

Tabla 5. Refactorizaciones aplicadas

N.º	Refactorización	Regla	Código refactorizado	Commit
1	Rename Variable	S1117	`JSONParser.peekStatus()`	3b5b819
2	Split Variable	S9364	`JSONParser.parse(Reader, ContainerFactory)`	af694a3
3	Remove Dead Code	S2737	`JSONParser.parse(Reader, ContainerFactory)`	76e9f09
4	Replace Nested Conditional with Guard Clauses	S3776	Los dos métodos `parse` de `JSONParser`	3b13c9d
5	Extract Method	S3776	Los dos métodos `parse` de `JSONParser`	5b73b90
6	Introduce Assertion	S131	`switch(status)` de `JSONParser`	8f916b4
7	Consolidate Duplicate Conditional Fragments	S1854	`JSONParser.parse(Reader, ContentHandler, boolean)`	0fda0d4
8	Introduce Special Case	S131	`Yytoken.toString()`	6d96bd9
9	Remove Setting Method	S1165	`ParseException`	bdc8a55
10	Agregar `@Override`	S1161	`ParseException`, `JSONObject`, `JSONArray`	1c12d6f
11	Hide Method (constructor)	S1118	`JSONValue`	21709ee
12	Move Method	S3776	`JSONValue.writeJSONString()` → `JSONArray`	170f7a3
13	Consolidate Conditional Expression	S3776	`JSONValue.writeJSONString()`	419d451
14	Decompose Conditional	S3776	`JSONValue.escape()`	320b441
15	Substitute Algorithm	S3776	`JSONValue.escape()`	36ad4e4

Cómo leer las figuras. Las figuras de código muestran líneas del repositorio con su número de línea real en esa versión. La etiqueta roja ANTES o verde DESPUÉS indica el momento. El código que interviene en la refactorización está resaltado: el mismo color y el mismo número identifican la misma pieza de código antes y después, y son los que se citan en los pasos (1, 2, 3, 4).

4.1 Refactorización 1: Rename Variable

Código	`JSONParser.peekStatus()`, que devuelve el estado que está en la cima de la pila del analizador. Commit `3b5b819`.
Problema	Regla `java:S1117`: la variable local `status` tiene el mismo nombre que el atributo `status` de la clase y lo oculta; al leer el método no es evidente cuál de los dos se usa.
Solución	Renombrar la variable local con un nombre que diga qué guarda.

La Figura 9 muestra el atributo (1), la variable local que lo oculta (2) y el resultado (3).

ANTES `JSONParser.java` atributo status y método peekStatus — paquete org.json.simple.parser

```
36    private int status = S_INIT;                                   (1) atributo de la clase
37
38    private int peekStatus(LinkedList statusStack){
39        if(statusStack.size()==0)
40            return -1;
41        Integer status=(Integer)statusStack.getFirst();            (2) variable local con el mismo nombre
42        return status.intValue();
43    }
```

DESPUÉS `JSONParser.java` método peekStatus — paquete org.json.simple.parser

```
38    private int peekStatus(LinkedList statusStack){
39        if(statusStack.size()==0)
40            return -1;
41        Integer topStatus=(Integer)statusStack.getFirst();         (3) nombre propio: estado en la cima de la pila
42        return topStatus.intValue();
43    }
```

Figura 9. Rename Variable: código antes y después de la refactorización

Pasos de la refactorización

1. Se identificó la variable local `status` (2), declarada en `peekStatus`, que oculta al atributo `status` (1).
2. Se eligió el nombre `topStatus`, porque la variable guarda el estado que está en la cima de la pila.
3. Se renombró la declaración y su único uso (3).
4. Se compiló el proyecto y se ejecutaron las 45 pruebas.

4.2 Refactorización 2: Split Variable

Código	`JSONParser.parse(Reader, ContainerFactory)`, que construye los objetos y arreglos a partir del texto JSON. Commit `af694a3`.
Problema	Regla `java:S9364` (3 casos): las variables `key` y `parent` se declaran en un grupo `case` y se reutilizan en otros grupos para guardar datos distintos; lo mismo ocurre con `val` en el estado `S_IN_ARRAY`. Cada variable tiene más de una responsabilidad.
Solución	Declarar una variable propia en cada grupo `case`.

En la Figura 10 se ven las declaraciones (1) y las reasignaciones en otros grupos `case` (2).

ANTES `JSONParser.java` parse(Reader, ContainerFactory): estados S_PASSED_PAIR_KEY y S_IN_ARRAY — paquete org.json.simple.parser

```
179                 case S_PASSED_PAIR_KEY:
180                     switch(token.type){
181                     case Yytoken.TYPE_COLON:
182                         break;
183                     case Yytoken.TYPE_VALUE:
184                         statusStack.removeFirst();
185                         String key=(String)valueStack.removeFirst();          (1) se declaran en este grupo case
186                         Map parent=(Map)valueStack.getFirst();
187                         parent.put(key,token.value);
188                         status=peekStatus(statusStack);
189                         break;
190                     case Yytoken.TYPE_LEFT_SQUARE:
191                         statusStack.removeFirst();
192                         key=(String)valueStack.removeFirst();                 (2) se reutilizan con otro dato
193                         parent=(Map)valueStack.getFirst();
194                         List newArray=createArrayContainer(containerFactory);
195                         parent.put(key,newArray);
196                         status=S_IN_ARRAY;
197                         statusStack.addFirst(Integer.valueOf(status));
198                         valueStack.addFirst(newArray);
199                         break;
200                     case Yytoken.TYPE_LEFT_BRACE:
201                         statusStack.removeFirst();
202                         key=(String)valueStack.removeFirst();                 (2) y otra vez
203                         parent=(Map)valueStack.getFirst();
204                         Map newObject=createObjectContainer(containerFactory);
205                         parent.put(key,newObject);
206                         status=S_IN_OBJECT;
207                         statusStack.addFirst(Integer.valueOf(status));
208                         valueStack.addFirst(newObject);
                    ⋮ líneas 209 a 218 omitidas
219                     case Yytoken.TYPE_VALUE:
220                         List val=(List)valueStack.getFirst();                 (1) declarada aquí
221                         val.add(token.value);
                    ⋮ líneas 222 a 232 omitidas
233                     case Yytoken.TYPE_LEFT_BRACE:
234                         val=(List)valueStack.getFirst();                      (2) reutilizada
235                         Map newObject=createObjectContainer(containerFactory);
236                         val.add(newObject);
                    ⋮ líneas 237 a 240 omitidas
241                     case Yytoken.TYPE_LEFT_SQUARE:
242                         val=(List)valueStack.getFirst();                      (2) reutilizada
243                         List newArray=createArrayContainer(containerFactory);
244                         val.add(newArray);
```

Figura 10. Split Variable: código antes de la refactorización

Pasos de la refactorización

1. Se ubicaron las declaraciones (1) y cada reasignación en otro grupo `case` (2).
2. En cada grupo que reasignaba una variable se declaró una nueva, con un nombre que dice qué guarda (3): `arrayKey` y `arrayParent` cuando el valor nuevo es un arreglo, `objectKey` y `objectParent` cuando es un objeto.
3. Se reemplazaron los usos dentro de cada grupo por la variable nueva.
4. Se compiló y se ejecutaron las pruebas.

En la Figura 11 cada variable se asigna una sola vez y su nombre indica en qué contenedor se agrega el valor nuevo.

DESPUÉS `JSONParser.java` parse(Reader, ContainerFactory): una variable por grupo case — paquete org.json.simple.parser

```
179                 case S_PASSED_PAIR_KEY:
180                     switch(token.type){
181                     case Yytoken.TYPE_COLON:
182                         break;
183                     case Yytoken.TYPE_VALUE:
184                         statusStack.removeFirst();
185                         String key=(String)valueStack.removeFirst();
186                         Map parent=(Map)valueStack.getFirst();
187                         parent.put(key,token.value);
188                         status=peekStatus(statusStack);
189                         break;
190                     case Yytoken.TYPE_LEFT_SQUARE:
191                         statusStack.removeFirst();
192                         String arrayKey=(String)valueStack.removeFirst();          (3) variables propias del grupo
193                         Map arrayParent=(Map)valueStack.getFirst();
194                         List newArray=createArrayContainer(containerFactory);
195                         arrayParent.put(arrayKey,newArray);
196                         status=S_IN_ARRAY;
197                         statusStack.addFirst(Integer.valueOf(status));
198                         valueStack.addFirst(newArray);
199                         break;
200                     case Yytoken.TYPE_LEFT_BRACE:
201                         statusStack.removeFirst();
202                         String objectKey=(String)valueStack.removeFirst();         (3) variables propias del grupo
203                         Map objectParent=(Map)valueStack.getFirst();
204                         Map newObject=createObjectContainer(containerFactory);
205                         objectParent.put(objectKey,newObject);
206                         status=S_IN_OBJECT;
207                         statusStack.addFirst(Integer.valueOf(status));
208                         valueStack.addFirst(newObject);
                                    ⋮ líneas 209 a 218 omitidas
219                     case Yytoken.TYPE_VALUE:
220                         List val=(List)valueStack.getFirst();
221                         val.add(token.value);
                                    ⋮ líneas 222 a 232 omitidas
233                     case Yytoken.TYPE_LEFT_BRACE:
234                         List objectParent=(List)valueStack.getFirst();             (3) variable propia
235                         Map newObject=createObjectContainer(containerFactory);
236                         objectParent.add(newObject);
                                    ⋮ líneas 237 a 240 omitidas
241                     case Yytoken.TYPE_LEFT_SQUARE:
242                         List arrayParent=(List)valueStack.getFirst();              (3) variable propia
243                         List newArray=createArrayContainer(containerFactory);
244                         arrayParent.add(newArray);
```

Figura 11. Split Variable: código después de la refactorización

4.3 Refactorización 3: Remove Dead Code

Código	`JSONParser.parse(Reader, ContainerFactory)`. Commit `76e9f09`.
Problema	Regla `java:S2737`: el método envolvía todo su bucle en un `try` cuyo único `catch` volvía a lanzar la misma `IOException`. Ese bloque no hace nada: la excepción se propaga igual sin él.
Solución	Eliminar el `try` y el `catch`.

ANTES `JSONParser.java` parse(Reader, ContainerFactory) — paquete org.json.simple.parser

```
113         LinkedList statusStack = new LinkedList();
114         LinkedList valueStack = new LinkedList();
115
116         try{                                          (1) try sin ningún propósito
117             do{
118                 nextToken();
                ⋮ líneas 119 a 258 omitidas
259             }while(token.type!=Yytoken.TYPE_EOF);
260         }
(1) catch que solo vuelve a lanzar la excepción
261         catch(IOException ie){
262             throw ie;
263         }
264
265         throw new ParseException(getPosition(), ParseException.ERROR_UNEXPECTED_TOKEN, token);
266     }
```

DESPUÉS `JSONParser.java` parse(Reader, ContainerFactory) — paquete org.json.simple.parser

```
113         LinkedList statusStack = new LinkedList();
114         LinkedList valueStack = new LinkedList();
115
116         do{                                           (3) el bucle ya no está dentro de un try
117             nextToken();
                ⋮ líneas 118 a 257 omitidas
258         }while(token.type!=Yytoken.TYPE_EOF);         (3) la IOException se propaga igual que antes
259
260         throw new ParseException(getPosition(), ParseException.ERROR_UNEXPECTED_TOKEN, token);
261     }
```

Figura 12. Remove Dead Code: código antes y después de la refactorización

Pasos de la refactorización

1. Se comprobó que el `catch` (1) no registra, transforma ni libera nada: solo hace `throw ie`.
2. Se comprobó que el método ya declara `throws IOException`, así que la excepción puede salir sin ese bloque.
3. Se eliminaron el `try` y el `catch`; el cuerpo del bucle perdió un nivel de sangría.
4. Se compiló y se ejecutaron las pruebas.

4.4 Refactorización 4: Replace Nested Conditional with Guard Clauses

Código	El estado `S_IN_FINISHED_VALUE` de los dos métodos `parse` de `JSONParser`. Commit `3b13c9d`.
Problema	Regla `java:S3776`: el `if/else` ponía al mismo nivel el camino normal (el texto terminó) y el caso de error. Cada `else` suma complejidad cognitiva y obliga a leer las dos ramas para saber cuál es la normal.
Solución	Revisar primero el caso de error con una cláusula de guarda y dejar el camino normal sin `else`.

ANTES `JSONParser.java` estado S_IN_FINISHED_VALUE en los dos métodos parse — paquete org.json.simple.parser

```
141             case S_IN_FINISHED_VALUE:
1 if/else: el caso normal y el error al mismo nivel
142                 if(token.type==Yytoken.TYPE_EOF)
143                     return valueStack.removeFirst();
144                 else
145                     throw new ParseException(getPosition(), ParseException.ERROR_UNEXPECTED_TOKEN, token);
                                        ⋮ líneas 146 a 368 omitidas
369                 case S_IN_FINISHED_VALUE:
370                     nextToken();
2 el mismo patrón en el análisis por eventos
371                     if(token.type==Yytoken.TYPE_EOF){
372                         contentHandler.endJSON();
373                         status = S_END;
374                         return;
375                     }
376                     else{
377                         status = S_IN_ERROR;
378                         throw new ParseException(getPosition(), ParseException.ERROR_UNEXPECTED_TOKEN,
    token);
379                     }
```

DESPUÉS `JSONParser.java` estado S_IN_FINISHED_VALUE en los dos métodos parse — paquete org.json.simple.parser

```
141             case S_IN_FINISHED_VALUE:
1 cláusula de guarda: el error sale primero
142                 if(token.type!=Yytoken.TYPE_EOF)
143                     throw new ParseException(getPosition(), ParseException.ERROR_UNEXPECTED_TOKEN, token);
144                 return valueStack.removeFirst();                    1 camino normal, sin else
                                        ⋮ líneas 145 a 367 omitidas
368                 case S_IN_FINISHED_VALUE:
369                     nextToken();
2 cláusula de guarda
370                     if(token.type!=Yytoken.TYPE_EOF){
371                         status = S_IN_ERROR;
372                         throw new ParseException(getPosition(), ParseException.ERROR_UNEXPECTED_TOKEN,
    token);
373                     }
374                     contentHandler.endJSON();                       2 camino normal, sin else
375                     status = S_END;
376                     return;
```

Figura 13. Replace Nested Conditional with Guard Clauses: código antes y después

Pasos de la refactorización

1. Se identificó en cada `if/else` la rama de error, la que se ejecuta cuando el token no es el fin del texto (1) (2).
2. Se invirtió la condición (`==` pasó a `!=`) para que la rama de error vaya primero y termine con `throw`.
3. Como la guarda siempre termina el método, el camino normal quedó después del `if`, sin `else`
4. Se compiló y se ejecutaron las pruebas.

4.5 Refactorización 5: Extract Method

Código	Los dos métodos `parse` de `JSONParser`: `parse(Reader, ContainerFactory)`, con complejidad 37, y `parse(Reader, ContentHandler, boolean)`, con complejidad 100. Commit `5b73b90`.
Problema	Regla `java:S3776`: cada método es una máquina de estados con un `switch` principal y, dentro de cada estado, otro `switch` por tipo de token. Todo está en un solo método de más de cien líneas, con cuatro niveles de anidamiento.
Solución	Pasar el código de cada estado a un método propio, con un nombre que diga qué estado procesa.

La Figura 14 muestra el método de análisis por eventos antes del cambio: el estado `S_INIT` (1) es uno de los ocho casos del `switch` principal (2).

ANTES `JSONParser.java` parse(Reader, ContentHandler, boolean) — paquete org.json.simple.parser

```
338         try{
339             do{
340                 switch(status){                    (2) switch principal: los 8 estados en un solo método
341                 case S_INIT:
```

(1) Fragmento 1: estado S_INIT, con un switch anidado

```
342                     contentHandler.startJSON();
343                     nextToken();
344                     switch(token.type){
345                     case Yytoken.TYPE_VALUE:
346                         status=S_IN_FINISHED_VALUE;
347                         statusStack.addFirst(Integer.valueOf(status));
348                         if(!contentHandler.primitive(token.value))
349                             return;                                (3) detiene el análisis
350                         break;
351                     case Yytoken.TYPE_LEFT_BRACE:
352                         status=S_IN_OBJECT;
353                         statusStack.addFirst(Integer.valueOf(status));
354                         if(!contentHandler.startObject())
355                             return;
356                         break;
357                     case Yytoken.TYPE_LEFT_SQUARE:
358                         status=S_IN_ARRAY;
359                         statusStack.addFirst(Integer.valueOf(status));
360                         if(!contentHandler.startArray())
361                             return;
362                         break;
363                     default:
364                         status=S_IN_ERROR;
365                     }//inner switch
366                     break;
                    : líneas 367 a 498 omitidas: los otros 7 estados, con la misma estructura
499                 }//switch
500                 if(status==S_IN_ERROR){
501                     throw new ParseException(getPosition(), ParseException.ERROR_UNEXPECTED_TOKEN, token);
502                 }
503             }while(token.type!=Yytoken.TYPE_EOF);
```

Figura 14. Extract Method: parse(Reader, ContentHandler, boolean) antes de la refactorización

Pasos de la refactorización

1. Se identificó un fragmento por estado. En el análisis por eventos cada fragmento puede detener el análisis con `return;` (3) cuando el `ContentHandler` lo pide.
2. Cada fragmento pasó a un método privado `handle<Estado>`, por ejemplo `handleInit` (1), que devuelve `boolean`: `false` donde antes había `return;` y `true` donde había `break;`.
3. Se creó `handleStatus` (2), que decide qué método llamar según el estado; en `parse` el `switch` principal se reemplazó por una sola llamada.
4. En `parse(Reader, ContainerFactory)` se hizo lo mismo con métodos `read<Estado>` (4); como ahí no hace falta detener el análisis, devuelven `void`.
5. Se compiló y se ejecutaron las pruebas, entre ellas las que detienen y reanudan el análisis con `isResume`.

DESPUÉS `JSONParser.java` parse(Reader, ContentHandler, boolean) y el método nuevo handleStatus — paquete org.json.simple.parser

```
354         try{
355             do{
356                 if(!handleStatus(contentHandler, statusStack))
357                     return;
358                 if(status==S_IN_ERROR){
359                     throw new ParseException(getPosition(), ParseException.ERROR_UNEXPECTED_TOKEN, token);
360                 }
361             }while(token.type!=Yytoken.TYPE_EOF);
362         }
```

2 reemplaza al switch principal

: líneas 363 a 383 omitidas

2 Método nuevo: decide qué estado procesar

```
384     /**
385      * Processes the current status of the stream parsing.
386      *
387      * @return false if the parsing must stop: the handler asked to stop or the JSON text has ended.
388      */
389     private boolean handleStatus(ContentHandler contentHandler, LinkedList statusStack) throws IOException,
    ParseException{
390         switch(status){
391         case S_INIT:
392             return handleInit(contentHandler, statusStack);
393         case S_IN_FINISHED_VALUE:
394             return handleFinishedValue(contentHandler);
395         case S_IN_OBJECT:
396             return handleInObject(contentHandler, statusStack);
397         case S_PASSED_PAIR_KEY:
398             return handlePassedPairKey(contentHandler, statusStack);
399         case S_IN_PAIR_VALUE:
400             return handleInPairValue(contentHandler, statusStack);
401         case S_IN_ARRAY:
402             return handleInArray(contentHandler, statusStack);
403         case S_END:
404             return false;
405         case S_IN_ERROR:
406             throw new ParseException(getPosition(), ParseException.ERROR_UNEXPECTED_TOKEN, token);
407         }
408         return true;
409     }
```

Figura 15. Extract Method: parse(Reader, ContentHandler, boolean) y el método nuevo handleStatus

DESPUÉS **JSONParser.java** método extraído handleInit — paquete org.json.simple.parser

1 Método extraído del fragmento 1

```
411     private boolean handleInit(ContentHandler contentHandler, LinkedList statusStack) throws IOException,
    ParseException{
412         contentHandler.startJSON();
413         nextToken();
414         switch(token.type){
415         case Yytoken.TYPE_VALUE:
416             status=S_IN_FINISHED_VALUE;
417             statusStack.addFirst(Integer.valueOf(status));
418             if(!contentHandler.primitive(token.value))
419                 return false;                                              3 antes: return;
420             break;
421         case Yytoken.TYPE_LEFT_BRACE:
422             status=S_IN_OBJECT;
423             statusStack.addFirst(Integer.valueOf(status));
424             if(!contentHandler.startObject())
425                 return false;
426             break;
427         case Yytoken.TYPE_LEFT_SQUARE:
428             status=S_IN_ARRAY;
429             statusStack.addFirst(Integer.valueOf(status));
430             if(!contentHandler.startArray())
431                 return false;
432             break;
433         default:
434             status=S_IN_ERROR;
435         }
436         return true;                                                       3 antes: break;
437     }
```

Figura 16. Extract Method: método extraído del fragmento 1

DESPUÉS `JSONParser.java` parse(Reader, ContainerFactory): cada estado en su método read... paquete org.json.simple.parser

```
118             switch(status){
119             case S_INIT:
120                 readInit(statusStack, valueStack, containerFactory);        4 métodos extraídos
121                 break;
122
123             case S_IN_FINISHED_VALUE:
124                 if(token.type!=Yytoken.TYPE_EOF)
125                     throw new ParseException(getPosition(), ParseException.ERROR_UNEXPECTED_TOKEN, token);
126                 return valueStack.removeFirst();
127
128             case S_IN_OBJECT:
129                 readInObject(statusStack, valueStack);
130                 break;
131
132             case S_PASSED_PAIR_KEY:
133                 readPassedPairKey(statusStack, valueStack, containerFactory);
134                 break;
135
136             case S_IN_ARRAY:
137                 readInArray(statusStack, valueStack, containerFactory);
138                 break;
139             case S_IN_ERROR:
140                 throw new ParseException(getPosition(), ParseException.ERROR_UNEXPECTED_TOKEN, token);
141             }//switch
```

Figura 17. Extract Method: parse(Reader, ContainerFactory) después de la refactorización

La Tabla 6 lista los métodos que se extrajeron de cada `parse`.

Tabla 6. Métodos extraídos de JSONParser

Método original	Métodos extraídos
`parse(Reader, ContainerFactory)`	`readInit`, `readInObject`, `readPassedPairKey`, `readInArray`
`parse(Reader, ContentHandler, boolean)`	`handleStatus`, `handleInit`, `handleFinishedValue`, `handleInObject`, `handlePassedPairKey`, `handleInPairValue`, `handleInArray`

4.6 Refactorización 6: Introduce Assertion

Código	Los `switch(status)` de `parse(Reader, ContainerFactory)` y de `handleStatus`. Commit `8f916b4`.
Problema	Regla `java:S131`: los `switch` no tenían `default`. El código supone que `status` siempre es uno de los estados conocidos, pero esa suposición no estaba escrita: si apareciera otro valor, el análisis seguiría sin avisar.
Solución	Agregar un `default` que lance `IllegalStateException`, para que la suposición quede explícita y un error de programación se detecte de inmediato.

ANTES `JSONParser.java` switch(status) de parse(Reader, ContainerFactory) y de handleStatus — paquete org.json.simple.parser

```
139             case S_IN_ERROR:
140                 throw new ParseException(getPosition(), ParseException.ERROR_UNEXPECTED_TOKEN, token);
141             }//switch                                                          (1) sin default
                                    : líneas 142 a 404 omitidas
405         case S_IN_ERROR:
406             throw new ParseException(getPosition(), ParseException.ERROR_UNEXPECTED_TOKEN, token);
407         }
408         return true;                                   (1) un estado desconocido seguiría en silencio
409     }
```

DESPUÉS `JSONParser.java` switch(status) de parse(Reader, ContainerFactory) y de handleStatus — paquete org.json.simple.parser

```
139             case S_IN_ERROR:
140                 throw new ParseException(getPosition(), ParseException.ERROR_UNEXPECTED_TOKEN, token);
(1) default: la suposición queda explícita
141             default:
142                 throw new IllegalStateException("Unexpected parser status: " + status);
143             }//switch
                                    : líneas 144 a 406 omitidas
407         case S_IN_ERROR:
408             throw new ParseException(getPosition(), ParseException.ERROR_UNEXPECTED_TOKEN, token);
409         default:
410             throw new IllegalStateException("Unexpected parser status: " + status);
411         }
412     }
```

Figura 18. Introduce Assertion: código antes y después de la refactorización

Pasos de la refactorización

1. Se revisaron los valores que puede tomar `status` en cada método: en `parse(Reader, ContainerFactory)` nunca toma `S_END` ni `S_IN_PAIR_VALUE`, y `handleStatus` ya cubre los ocho estados.
2. Se agregó en ambos `switch` un `default` (1) que lanza `IllegalStateException` con el estado inesperado.
3. En `handleStatus` se eliminó el `return true;` final, que ya no se puede alcanzar.
4. Se compiló y se ejecutaron las pruebas: ninguna llega al `default`, lo que confirma que el comportamiento no cambió.

4.7 Refactorización 7: Consolidate Duplicate Conditional Fragments

Código	Inicio de `JSONParser.parse(Reader, ContentHandler, boolean)`, que decide si empieza un análisis nuevo o reanuda el anterior. Commit `0fda0d4`.
Problema	Regla `java:S1854`: la asignación `isResume = false` no sirve, porque la variable no se vuelve a leer. Además, las dos ramas del `if/else` repiten el mismo fragmento: reiniciar el lector y crear la pila de estados.
Solución	Dejar el fragmento una sola vez, bajo una condición que reúne los dos casos en que se ejecutaba.

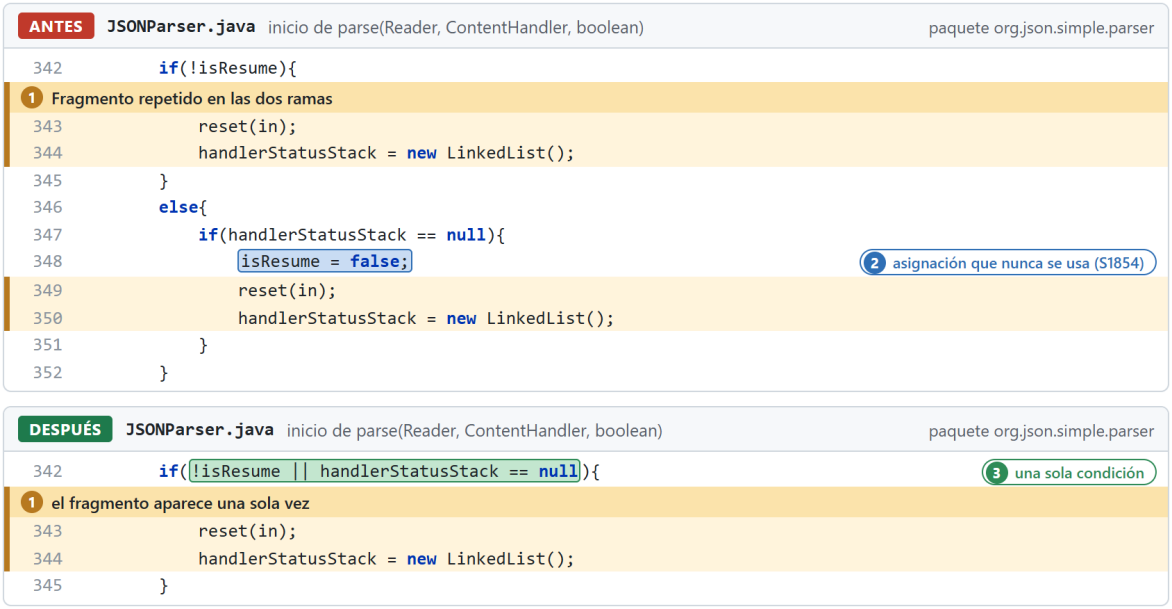


Figura 19. Consolidate Duplicate Conditional Fragments: código antes y después

Pasos de la refactorización

1. Se identificó el fragmento repetido (1) en las dos ramas y la asignación inútil (2).
2. Se comprobó cuándo se ejecuta el fragmento: cuando no se reanuda (`!isResume`) o cuando se pide reanudar pero no hay un análisis previo (`handlerStatusStack == null`).
3. Se unieron esos dos casos en una sola condición (3) y el fragmento quedó una sola vez; la asignación inútil desapareció junto con la rama `else`.
4. Se compiló y se ejecutaron las pruebas, incluida la que reanuda el análisis.

4.8 Refactorización 8: Introduce Special Case

Código	`Yytoken.toString()`, que describe un token del analizador léxico. Commit `6d96bd9`.
Problema	Regla `java:S131`: el `switch` sobre el tipo de token no tenía `default`. Un tipo desconocido era un caso especial que el código manejaba de forma implícita: devolvía una cadena vacía sin que nada lo indicara.
Solución	Escribir ese caso especial de forma explícita en un `default`, con el mismo resultado que antes.

ANTES `Yytoken.java` método toString — paquete org.json.simple.parser

```
53          case TYPE_COLON:
54              sb.append("COLON(:)");
55              break;
56          case TYPE_EOF:
57              sb.append("END OF FILE");
58              break;
59          }                                   (1) sin default: un tipo desconocido queda implícito
60          return sb.toString();
61      }
```

DESPUÉS `Yytoken.java` método toString — paquete org.json.simple.parser

```
53          case TYPE_COLON:
54              sb.append("COLON(:)");
55              break;
56          case TYPE_EOF:
57              sb.append("END OF FILE");
58              break;
(1) caso especial explícito: tipo desconocido, misma salida que antes
59          default:
60              // Unknown token type: it has no description, as before.
61              break;
62          }
63          return sb.toString();
64      }
```

Figura 20. Introduce Special Case: código antes y después de la refactorización

Pasos de la refactorización

1. Se comprobó qué devuelve el método para un tipo desconocido: una cadena vacía, porque ningún `case` agrega texto.
2. Se agregó un `default` (1) con un comentario que documenta ese caso y que conserva la cadena vacía, para no cambiar el comportamiento.
3. Se compiló y se ejecutaron las pruebas.

4.9 Refactorización 9: Remove Setting Method

Código	`ParseException`, la excepción que indica dónde y por qué falla el análisis. Commit `bdc8a55`.
Problema	Regla `java:S1165` (3 casos): los atributos de una excepción deberían ser `final`, porque la excepción describe un error que ya ocurrió y no debería cambiar. Aquí no podían serlo porque la clase tenía un método `set` para cada atributo, aunque ningún código del proyecto los usa.
Solución	Eliminar los métodos `set` y declarar los atributos como `final`.

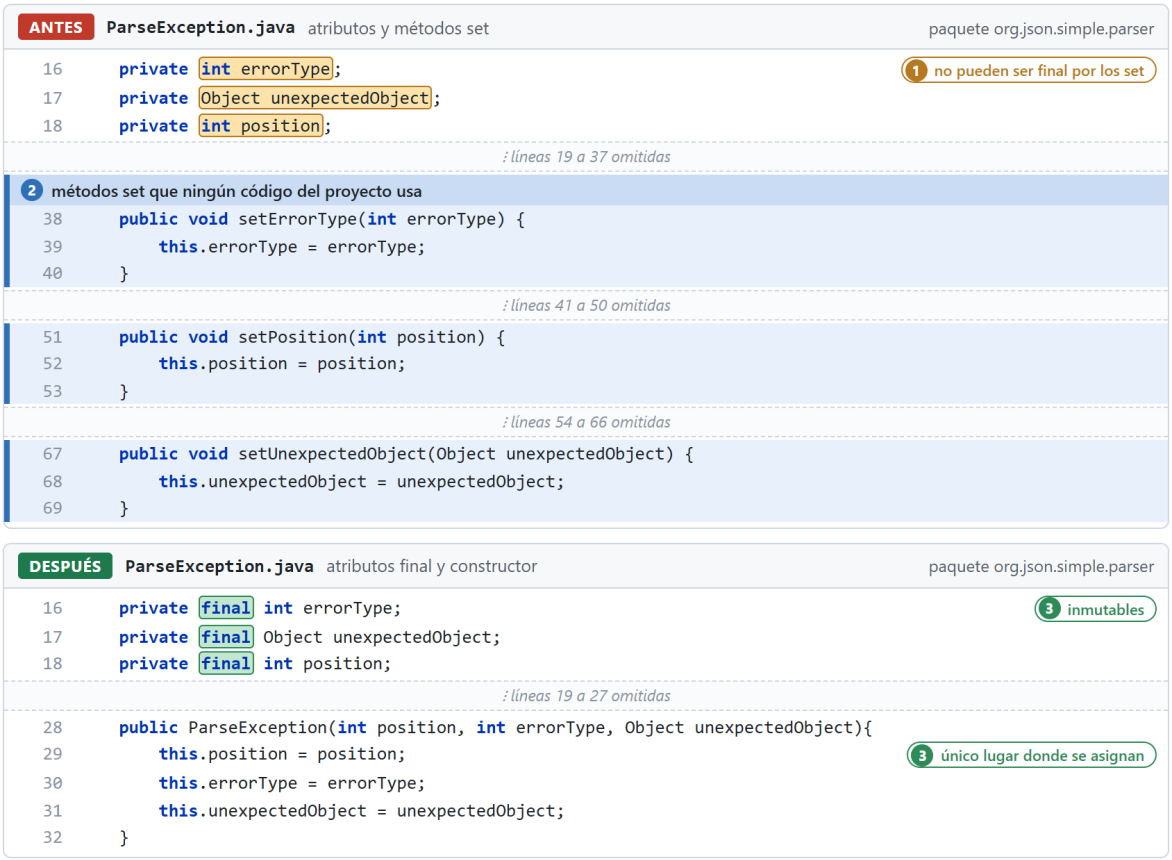


Figura 21. Remove Setting Method: código antes y después de la refactorización

Pasos de la refactorización

1. Se buscó en todo el proyecto, incluidas las pruebas, cada uso de `setErrorType`, `setPosition` y `setUnexpectedObject` (2): no hay ninguno.
2. Se eliminaron los tres métodos `set`.
3. Se declararon los tres atributos (1) como `final` (3); el compilador confirmó que solo se asignan en el constructor.
4. Se compiló y se ejecutaron las pruebas.

Eliminar un método público cambia la interfaz de la clase; se aceptó porque ningún código del proyecto lo usa y porque una excepción que no cambia después de crearse es más fácil de razonar.

4.10 Refactorización 10: Agregar @Override

Código	`ParseException.toString()`, `ParseException.getMessage()`, `JSONObject.toString()` y `JSONArray.toString()`. Commit `1c12d6f`.
Problema	Regla `java:S1161` (4 casos): estos métodos sobrescriben métodos heredados pero no lo declaran. Sin la anotación, si alguien cambia la firma por error, el compilador no avisa y el método deja de sobrescribir sin que nadie lo note.
Solución	Agregar la anotación `@Override` a cada método.

Esta refactorización no forma parte del catálogo de Fowler; es una práctica propia de Java que deja explícito el diseño y que el compilador verifica.

ANTES `ParseException.java` toString y getMessage — paquete org.json.simple.parser

```
64      public String toString() {                    1 sobrescribe Throwable.toString
65          return getMessage();
66      }
67
68      public String getMessage() {                  1 sobrescribe Throwable.getMessage
```

DESPUÉS `ParseException.java` toString y getMessage — paquete org.json.simple.parser

```
64      @Override
65      public String toString() {
66          return getMessage();
67      }
68
69      @Override
70      public String getMessage() {
```

DESPUÉS `JSONObject.java` toString — paquete org.json.simple

```
102     @Override
103     public String toString(){
104         return toJSONString();
105     }
```

DESPUÉS `JSONArray.java` toString — paquete org.json.simple

```
407     @Override
408     public String toString() {
409         return toJSONString();
410     }
```

Figura 22. Agregar @Override: código antes y después de la refactorización

Pasos de la refactorización

1. Se identificaron los métodos que sobrescriben `toString()` y `getMessage()` heredados 1.
2. Se agregó `@Override` sobre cada uno 3.
3. Se compiló: el compilador confirma que cada método marcado realmente sobrescribe uno heredado.
4. Se ejecutaron las pruebas.

4.11 Refactorización 11: Hide Method

Código	`JSONValue`, una clase utilitaria con métodos `static` para leer y escribir JSON. Commit `21709ee`.
Problema	Regla `java:S1118`: la clase solo tiene métodos `static`, pero Java le da un constructor público implícito, así que se puede crear un objeto `JSONValue` que no sirve para nada.
Solución	Declarar un constructor `private` para ocultar el constructor público.

ANTES `JSONValue.java` declaración de la clase — paquete org.json.simple

```
20 /**
21  * @author FangYidong &lt;fangyidong@gmail.com&gt;
22  */
23 public class JSONValue {                    1 solo métodos static, pero se puede instanciar
24     /**
25      * Parse JSON text into java object from the input source.
```

DESPUÉS `JSONValue.java` declaración de la clase — paquete org.json.simple

```
20 /**
21  * @author FangYidong &lt;fangyidong@gmail.com&gt;
22  */
23 public class JSONValue {
   1 constructor privado: oculta el constructor público implícito
24     /**
25      * Utility class: all its methods are static, so it is not instantiated.
26      */
27     private JSONValue(){
28         // No instances.
29     }
30
31     /**
```

Figura 23. Hide Method: código antes y después de la refactorización

Pasos de la refactorización

1. Se comprobó que ningún código del proyecto crea objetos `JSONValue` ni extiende la clase.
2. Se agregó un constructor privado vacío (1), con un comentario que explica por qué existe.
3. Se compiló y se ejecutaron las pruebas.

4.12 Refactorización 12: Move Method

Código	`JSONValue.writeJSONString(Object, Writer)`, que escribe cualquier valor como texto JSON. Commit `170f7a3`.
Problema	Regla `java:S3776`: el método tenía complejidad 27 (el máximo es 15). Nueve de sus condiciones solo reparten los arreglos de Java (`byte[]`, `int[]`, …, `Object[]`) entre las sobrecargas de `JSONArray`, que es la clase que sabe escribir arreglos.
Solución	Llevar ese reparto a un método de `JSONArray`, junto a las sobrecargas que usa, y dejar en `writeJSONString` una sola llamada.

ANTES `JSONValue.java` método writeJSONString — paquete org.json.simple

```
184         if(value instanceof Collection){
185             JSONArray.writeJSONString((Collection)value, out);
186             return;
187         }
188
```

1 Fragmento: 9 condiciones, una por cada tipo de arreglo

```
189         if(value instanceof byte[]){
190             JSONArray.writeJSONString((byte[])value, out);
191             return;
192         }
193
194         if(value instanceof short[]){
195             JSONArray.writeJSONString((short[])value, out);
196             return;
197         }
198
```

: líneas 199 a 223 omitidas: int[], long[], float[], double[] y boolean[], con la misma forma

```
224         if(value instanceof char[]){
225             JSONArray.writeJSONString((char[])value, out);
226             return;
227         }
228
229         if(value instanceof Object[]){
230             JSONArray.writeJSONString((Object[])value, out);
231             return;
232         }
233
234         out.write(value.toString());
235     }
```

Figura 24. Move Method: código antes de la refactorización

Pasos de la refactorización

1. Se identificó el fragmento 1: nueve `if` seguidos, uno por tipo de arreglo, que solo llaman a `JSONArray.writeJSONString`.
2. Se creó en `JSONArray` el método `writeJSONStringIfArray(Object, Writer)` con ese fragmento, en el mismo orden. Devuelve `true` si el valor era un arreglo y lo escribió, y `false` en caso contrario.
3. Dentro del método movido, las llamadas `JSONArray.writeJSONString(...)` pasaron a `writeJSONString(...)`, porque ahora están en la misma clase.
4. En `JSONValue.writeJSONString` el fragmento se reemplazó por una llamada al método movido 1.
5. Se compiló y se ejecutaron las pruebas.

DESPUÉS `JSONValue.java` método writeJSONString — paquete org.json.simple

```
184         if(value instanceof Collection){
185             JSONArray.writeJSONString((Collection)value, out);
186             return;
187         }
188
189         if(JSONArray.writeJSONStringIfArray(value, out)){      (1) delega en JSONArray
190             return;
191         }
192
193         out.write(value.toString());
```

DESPUÉS `JSONArray.java` método movido writeJSONStringIfArray — paquete org.json.simple

```
399     /**
400      * Writes the value as a JSON array if it is a Java array of one of the supported types.
401      *
402      * @param value the value to encode
403      * @param out the writer to write the JSON text to
404      * @return true if the value was an array and was written; false otherwise
405      * @throws IOException if the writer does
406      */
```

(1) Método movido: queda junto a las sobrecargas writeJSONString de arreglos

```
407     static boolean writeJSONStringIfArray(Object value, Writer out) throws IOException{
408         if(value instanceof byte[]){
409             writeJSONString((byte[])value, out);
410             return true;
411         }
412
413         if(value instanceof short[]){
414             writeJSONString((short[])value, out);
415             return true;
        : líneas 416 a 447 omitidas: int[], long[], float[], double[], boolean[] y char[]
448         if(value instanceof Object[]){
449             writeJSONString((Object[])value, out);
450             return true;
451         }
452
453         return false;
454     }
```

Figura 25. Move Method: código después de la refactorización

4.13 Refactorización 13: Consolidate Conditional Expression

Código	`JSONValue.writeJSONString(Object, Writer)`. Commit `419d451`.
Problema	Regla `java:S3776`: además del reparto de arreglos, el método repetía condiciones con el mismo resultado: `Double` y `Float` se trataban en dos condiciones con la misma lógica (1), y `Number` y `Boolean` en dos condiciones que hacen lo mismo (2).
Solución	Unir cada par en una sola condición con `||`.

ANTES `JSONValue.java` método writeJSONString — paquete org.json.simple

1 Double y Float: la misma lógica en dos condiciones

```
143         if(value instanceof Double){
144             if(((Double)value).isInfinite() || ((Double)value).isNaN())
145                 out.write("null");
146             else
147                 out.write(value.toString());
148             return;
149         }
150
151         if(value instanceof Float){
152             if(((Float)value).isInfinite() || ((Float)value).isNaN())
153                 out.write("null");
154             else
155                 out.write(value.toString());
156             return;
157         }
158
```

2 Number y Boolean: el mismo resultado

```
159         if(value instanceof Number){
160             out.write(value.toString());
161             return;
162         }
163
164         if(value instanceof Boolean){
165             out.write(value.toString());
166             return;
167         }
```

DESPUÉS `JSONValue.java` método writeJSONString — paquete org.json.simple

1 una sola condición para Double y Float

```
143         if(value instanceof Double || value instanceof Float){
144             double number=((Number)value).doubleValue();
145             if(Double.isInfinite(number) || Double.isNaN(number))
146                 out.write("null");
147             else
148                 out.write(value.toString());
149             return;
150         }
151
```

2 una sola condición para Number y Boolean

```
152         if(value instanceof Number || value instanceof Boolean){
153             out.write(value.toString());
154             return;
155         }
```

Figura 26. Consolidate Conditional Expression: código antes y después

Pasos de la refactorización

1. Se comprobó que las ramas de `Double` y `Float` hacen lo mismo: escriben `null` si el número es infinito o NaN y, si no, su texto.
2. Se unieron en la condición `value instanceof Double || value instanceof Float` (1). Para revisar infinito y NaN el valor se convierte a `double`, conversión que conserva esos dos casos también para `Float`.
3. Se unieron las ramas de `Number` y `Boolean` (2), que escriben `value.toString()`. Se mantuvo el orden: `Double` y `Float` se revisan antes que `Number`.
4. Se compiló y se ejecutaron las pruebas. Como ninguna prueba escribe NaN ni infinito, la equivalencia se comprobó además con un script aparte (sección 6).

4.14 Refactorización 14: Decompose Conditional

Código	`JSONValue.escape(String, StringBuffer)`, que escapa los caracteres especiales de un texto. Commit `320b441`.
Problema	Regla `java:S3776`: el método tenía complejidad 16 (el máximo es 15). Buena parte venía de una condición con seis comparaciones (1) que, sin el comentario de arriba (2), no dice qué comprueba.
Solución	Extraer la condición a un método con un nombre que explique su propósito.

ANTES `JSONValue.java` método escape — paquete org.json.simple

```
256             default:
257                 //Reference: http://www.unicode.org/versions/Unicode5.1.0/        (2) comentario que explica la condición
258                 if((ch>='\u0000' && ch<='\u001F') || (ch>='\u007F' && ch<='\u009F') || (ch>='\u2000' &&
    ch<='\u20FF')){
259                     String ss=Integer.toHexString(ch);
260                     sb.append("\\u");
261                     for(int k=0;k<4-ss.length();k++){
262                         sb.append('0');
263                     }
264                     sb.append(ss.toUpperCase());
265                 }
266                 else{
267                     sb.append(ch);
268                 }
```

DESPUÉS `JSONValue.java` método escape y el método nuevo needsUnicodeEscape — paquete org.json.simple

```
256             default:
257                 if(needsUnicodeEscape(ch)){
258                     String ss=Integer.toHexString(ch);
259                     sb.append("\\u");
260                     for(int k=0;k<4-ss.length();k++){
261                         sb.append('0');
262                     }
263                     sb.append(ss.toUpperCase());
                                    : líneas 264 a 271 omitidas
(1) Método nuevo: la condición tiene un nombre que dice qué comprueba
272     /**
273      * Tells if a character must be written as a Unicode escape sequence: the control characters
274      * (U+0000 to U+001F and U+007F to U+009F) and the characters from U+2000 to U+20FF.
275      * Reference: http://www.unicode.org/versions/Unicode5.1.0/
276      */
277     private static boolean needsUnicodeEscape(char ch){
278         return (ch>='\u0000' && ch<='\u001F') || (ch>='\u007F' && ch<='\u009F') || (ch>='\u2000' &&
    ch<='\u20FF');
279     }
```

Figura 27. Decompose Conditional: código antes y después de la refactorización

Pasos de la refactorización

1. Se identificó la condición (1) y el comentario (2) que la explicaba.
2. Se creó el método privado `needsUnicodeEscape(char)`, que devuelve el resultado de la condición; el comentario pasó a su Javadoc.
3. La condición del `if` se reemplazó por la llamada al método.
4. Se compiló y se ejecutaron las pruebas.

4.15 Refactorización 15: Substitute Algorithm

Código	`JSONValue.escape(String, StringBuffer)`. Commit `36ad4e4`.
Problema	Regla `java:S3776`: para escribir un carácter como `\uXXXX` el método convertía el código a hexadecimal, agregaba ceros a la izquierda con un bucle `for` y pasaba las letras a mayúsculas (1). Son varios pasos y un bucle anidado para algo que Java resuelve con una sola llamada.
Solución	Reemplazar ese algoritmo por `String.format("\\u%04X", (int) ch)`, que produce el mismo texto.

ANTES `JSONValue.java` método escape — paquete org.json.simple

```
257                 if(needsUnicodeEscape(ch)){
(1) algoritmo manual: hexadecimal, ceros a la izquierda y mayúsculas
258                     String ss=Integer.toHexString(ch);
259                     sb.append("\\u");
260                     for(int k=0;k<4-ss.length();k++){
261                         sb.append('0');
262                     }
263                     sb.append(ss.toUpperCase());
264                 }
```

DESPUÉS `JSONValue.java` método escape — paquete org.json.simple

```
257                 if(needsUnicodeEscape(ch)){
(1) el mismo resultado con String.format("\u%04X")
258                     sb.append(String.format("\\u%04X", (int)ch));
259                 }
```

Figura 28. Substitute Algorithm: código antes y después de la refactorización

Pasos de la refactorización

1. Se comprobó qué produce el algoritmo original: `\u` seguido del código del carácter en hexadecimal, en mayúsculas y con cuatro dígitos.
2. Se reemplazaron las seis líneas (1) por una llamada a `String.format` con el formato `%04X`: hexadecimal en mayúsculas, con ceros a la izquierda hasta completar cuatro dígitos.
3. Se compiló y se ejecutaron las pruebas, que verifican el escape de caracteres de control.

5. Análisis del código v1.1

5.1 Resultados generales

Después de las 15 refactorizaciones se analizó la versión v1.1 con la misma configuración. La Figura 29 muestra el resumen y la Figura 30 la lista de proyectos con las dos versiones.

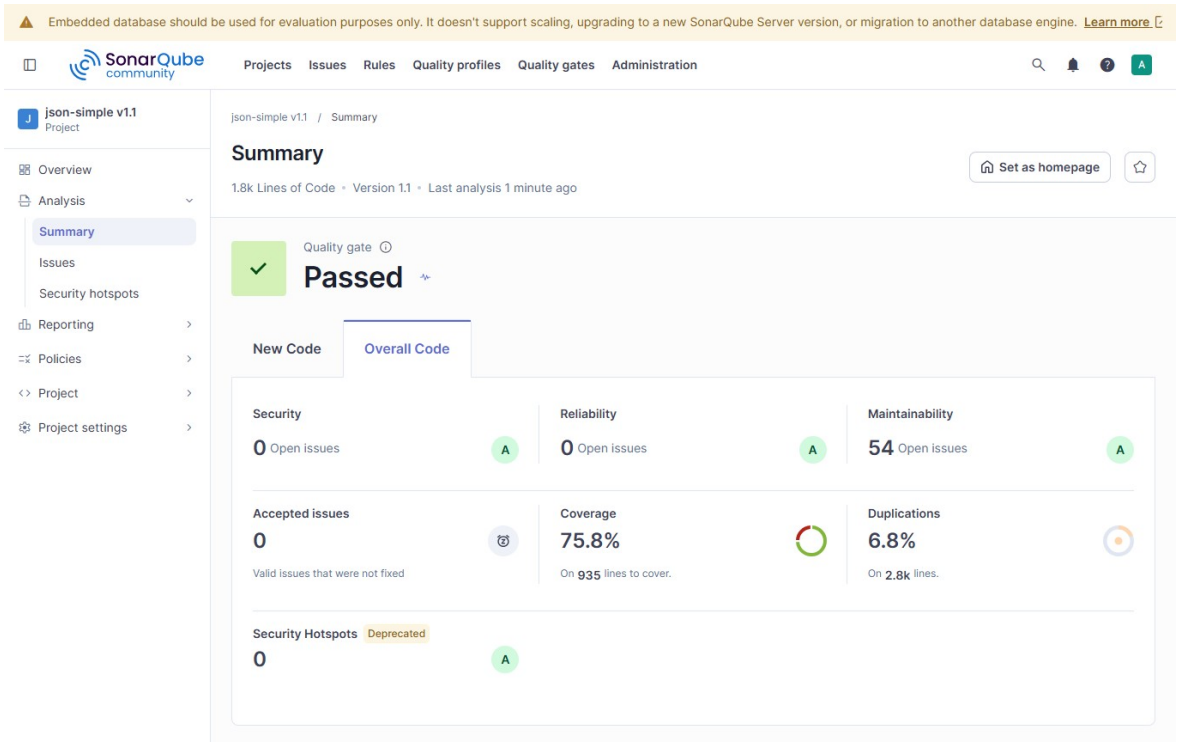


Figura 29. Resumen del análisis de la v1.1 en SonarQube

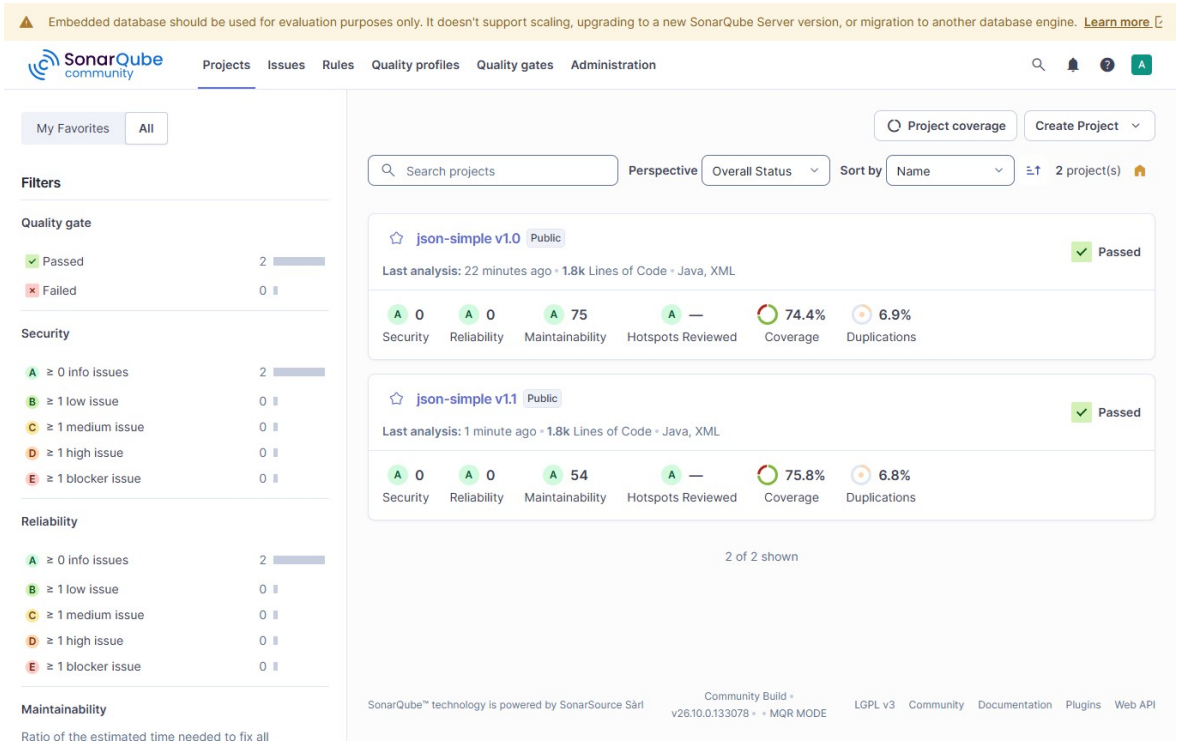


Figura 30. Las dos versiones analizadas en SonarQube

La Tabla 7 compara las métricas de las dos versiones.

Tabla 7. Comparación de las métricas de la v1.0 y la v1.1

Métrica	v1.0	v1.1	Cambio
Code smells (total)	75	54	-21 (−28,0 %)
Code smells en el código principal	48	27	-21
Code smells en el código escrito a mano	27	6	-21 (−77,8 %)
Deuda técnica (min)	872	606	-266 (−30,5 %)
Complejidad cognitiva	343	273	-70
Complejidad ciclomática	363	372	+9
Funciones	127	138	+11
Líneas de código	1 822	1 836	+14
Cobertura	74,4 %	75,8 %	+1,4 pp
Líneas duplicadas	6,9 %	6,8 %	−0,1 pp
Bugs / vulnerabilidades	0 / 0	0 / 0	-
Calificación de mantenibilidad	A	A	-
Quality Gate	Aprobado	Aprobado	-

Se resolvieron 21 de los 27 problemas del código escrito a mano y la deuda técnica bajó 266 minutos. La complejidad cognitiva bajó porque se eliminó anidamiento, mientras que la ciclomática subió un poco porque hay más métodos: cada método nuevo suma un punto, pero se entiende por separado. La cobertura subió porque se eliminó código que las pruebas no recorrían, como los métodos `set` de `ParseException` y el `catch` que solo relanzaba la excepción.

5.2 Resultados por archivo

La Tabla 8 muestra el cambio en cada archivo del código principal que tenía problemas.

Tabla 8. Cambios por archivo entre la v1.0 y la v1.1

Archivo	Complejidad cognitiva	Code smells	Deuda (min)	Funciones
`JSONParser.java`	145 → 82	12 → 2	190 → 22	15 → 26
`JSONValue.java`	46 → 30	3 → 0	28 → 0	8 → 10
`JSONArray.java`	62 → 71	3 → 2	25 → 20	27 → 28
`ParseException.java`	1 → 1	6 → 1	85 → 30	11 → 8
`JSONObject.java`	8 → 8	1 → 0	5 → 0	9 → 9
`Yytoken.java`	1 → 1	2 → 1	35 → 30	2 → 2
`Yylex.java` (generado)	63 → 63	21 → 21	251 → 251	21 → 21

`JSONParser` concentra la mayor mejora: su complejidad cognitiva bajó de 145 a 82 y ningún método supera ahora el límite de 15. En `JSONValue` bajó de 46 a 30 y quedó sin problemas; en `JSONArray` subió de 62 a 71 porque recibió el método movido en la refactorización 12, que reparte los arreglos sin anidamiento.

5.3 Problemas pendientes

En el código escrito a mano quedan 6 problemas (Figura 31 y Tabla 9). No se refactorizaron porque cambiarlos modificaría el comportamiento o la compatibilidad de la librería, o porque son menores.

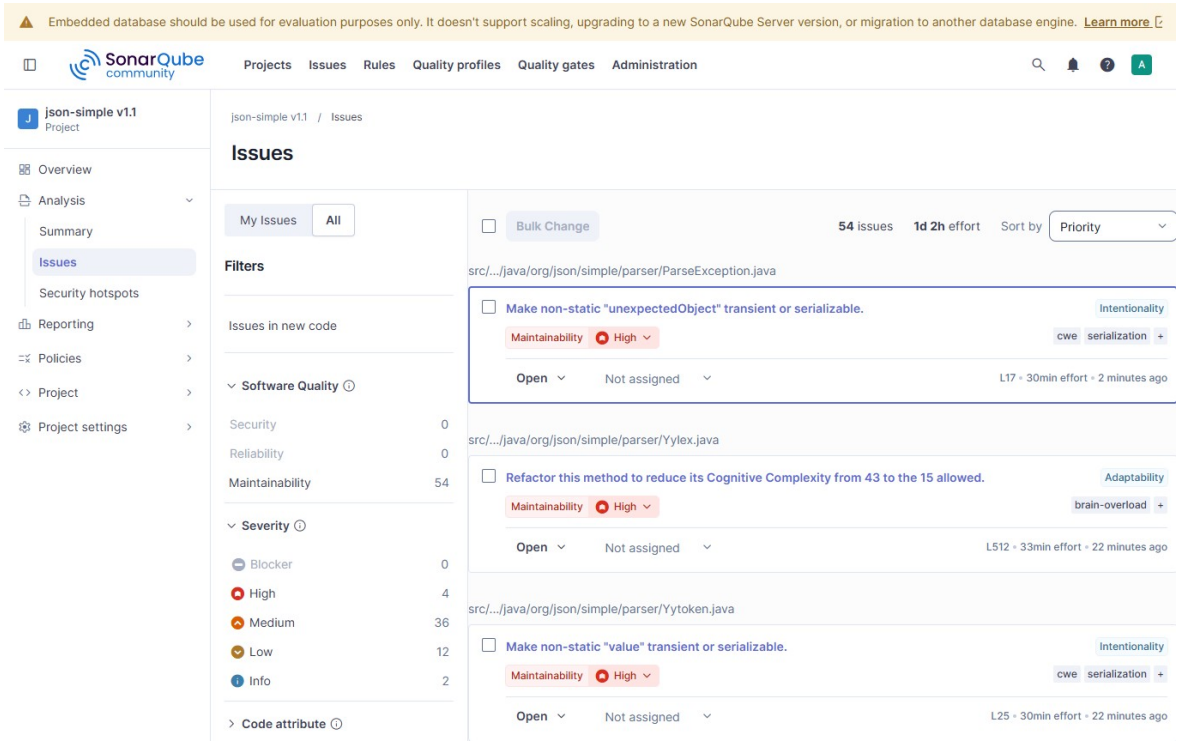


Figura 31. Lista de problemas de la v1.1

Tabla 9. Problemas pendientes y motivo

Regla	Ubicación	Motivo
`S1133` (2)	`JSONArray`	Los métodos `writeJSONString(List, Writer)` y `toJSONString(List)` están marcados como obsoletos a propósito: se conservan para que los programas compilados con la versión 1.1.1 sigan funcionando. Eliminarlos rompería esa compatibilidad binaria.
`S1948` (2)	`ParseException`, `Yytoken`	El atributo es de tipo `Object`. Marcarlo como `transient` cambiaría lo que se guarda al serializar la excepción o el token.
`S1181` (1)	`JSONParser`	El `catch` de `Error` existe porque el analizador léxico generado (`Yylex`) lanza `Error` ante fallos internos. Quitarlo cambiaría el estado en que queda el analizador después de ese fallo.
`S1155` (1)	`JSONParser.peekStatus()`	Cambio menor (`size()==0` por `isEmpty()`). Se dio prioridad a refactorizaciones de tipos diferentes; queda para una siguiente iteración.
21 problemas	`Yylex` (generado)	Código generado por JFlex: los cambios se perderían al regenerar la clase.

6. Verificación

Una refactorización es correcta solo si el programa sigue haciendo lo mismo [1]. Para comprobarlo se ejecutaron las 45 pruebas automatizadas del proyecto (JUnit 4) sobre el código original y sobre

cada uno de los 15 commits, cada vez en un directorio limpio y con el mismo contenedor de Maven (Tabla 10).

Tabla 10. Resultado de las pruebas antes de refactorizar y después de cada commit

Estado del código	Commit	Compilación	Pruebas exitosas
Antes de refactorizar (v1.0)	`47f9717`	Correcta	45 de 45
Refactorización 1: Rename Variable	`3b5b819`	Correcta	45 de 45
Refactorización 2: Split Variable	`af694a3`	Correcta	45 de 45
Refactorización 3: Remove Dead Code	`76e9f09`	Correcta	45 de 45
Refactorización 4: Replace Nested Conditional with Guard Clauses	`3b13c9d`	Correcta	45 de 45
Refactorización 5: Extract Method	`5b73b90`	Correcta	45 de 45
Refactorización 6: Introduce Assertion	`8f916b4`	Correcta	45 de 45
Refactorización 7: Consolidate Duplicate Conditional Fragments	`0fda0d4`	Correcta	45 de 45
Refactorización 8: Introduce Special Case	`6d96bd9`	Correcta	45 de 45
Refactorización 9: Remove Setting Method	`bdc8a55`	Correcta	45 de 45
Refactorización 10: Agregar `@Override`	`1c12d6f`	Correcta	45 de 45
Refactorización 11: Hide Method (constructor)	`21709ee`	Correcta	45 de 45
Refactorización 12: Move Method	`170f7a3`	Correcta	45 de 45
Refactorización 13: Consolidate Conditional Expression	`419d451`	Correcta	45 de 45
Refactorización 14: Decompose Conditional	`320b441`	Correcta	45 de 45
Refactorización 15: Substitute Algorithm	`36ad4e4`	Correcta	45 de 45

Las pruebas no escriben números NaN ni infinitos, que es justo lo que cambia la refactorización 13. Por eso, además, se ejecutó un mismo script de `jshell` sobre la v1.0 y la v1.1 con 32 valores de prueba (NaN e infinito de `Double` y `Float`, números, cadenas, arreglos, listas y mapas) y con el escape de los 65 536 caracteres del plano básico de Unicode. Las salidas de las dos versiones fueron idénticas.

7. Entregables y repositorio

El código está publicado en un repositorio público de GitHub:

https://github.com/JoelAlexanderProg/TG3_Equipo7

El historial tiene un commit con el código original, un commit por cada refactorización, con su nombre y la regla de SonarQube en el mensaje, y un commit final con la documentación. Las entregas corresponden a las etiquetas del repositorio (Tabla 11).

Tabla 11. Entregables

Entregable	Contenido	En el repositorio
`codigo_v1.0`	Código original de json-simple, sin refactorizar	Etiqueta `v1.0`
`codigo_v1.1`	Código después de las 15 refactorizaciones	Etiqueta `v1.1`
`TG3_Equipo7.pdf`	Este informe	Carpeta `informe/`

8. Conclusiones

- Levantar SonarQube en Docker requiere pocos comandos. La red permite que el análisis llegue al servidor por su nombre y los volúmenes conservan la información aunque se recree el contenedor.
- El perfil de calidad define qué se mide: con el perfil por defecto json-simple no mostraba problemas, y con *Sonar way comprehensive* aparecieron 75.
- Las 15 refactorizaciones resolvieron 21 de los 27 problemas del código escrito a mano, bajaron la deuda técnica de 872 a 606 minutos y la complejidad cognitiva de `JSONParser` de 145 a 82, sin cambiar el comportamiento: las 45 pruebas pasan en cada commit.
- Volver a analizar después de refactorizar es necesario: el segundo análisis confirma qué problemas se resolvieron y detecta si un cambio introdujo uno nuevo.
- No todo aviso debe corregirse. En el código generado o en métodos que existen por compatibilidad, el cambio cuesta más de lo que mejora.

APRENDIZAJE OBTENIDO

Antes de este trabajo conocíamos la teoría de la refactorización y los comandos básicos de Docker, pero no habíamos usado una herramienta de análisis estático. Al terminarlo sabemos levantar SonarQube en un contenedor con una red y volúmenes, generar un token para el escáner, ejecutar el análisis desde Maven en otro contenedor y medir la cobertura con JaCoCo.

También aprendimos a leer el reporte: diferenciar *bugs*, vulnerabilidades y *code smells*; entender qué es la deuda técnica y la complejidad cognitiva, y por qué un `switch` anidado dentro de otro pesa más que dos `switch` seguidos; y priorizar los problemas por severidad. Descubrimos que el perfil de calidad cambia por completo el resultado y que conviene revisarlo antes de sacar conclusiones.

Sobre la refactorización, aprendimos a trabajar en pasos pequeños, con un commit y una ejecución de las pruebas por cambio, y a elegir la refactorización según el problema: no es lo mismo una variable con dos responsabilidades que un método que mezcla ocho estados. Al volver a analizar descubrimos que la extracción de métodos había dejado un `if` que se podía simplificar, y lo corregimos dentro de la misma refactorización; eso nos mostró que el segundo análisis es parte del proceso. Por último, entendimos que las pruebas solo protegen lo que prueban: cuando una refactorización toca un caso sin pruebas, como los números NaN, hay que comprobar la equivalencia de otra forma.

REFERENCIAS

[1] M. Fowler, *Refactoring: Improving the Design of Existing Code*, 2nd ed. Boston, MA, USA: Addison-Wesley, 2018.

[2] SonarSource, "SonarQube Community Build documentation," 2026. [Online]. Available: https://docs.sonarsource.com/sonarqube-community-build/

[3] Docker Inc., "sonarqube: Docker Official Image," Docker Hub, 2026. [Online]. Available: https://hub.docker.com/_/sonarqube

[4] F. Yidong, "json-simple: A simple Java toolkit for JSON," GitHub, 2026. [Online]. Available: https://github.com/fangyidong/json-simple

[5] G. A. Campbell, "Cognitive Complexity: A new way of measuring understandability," SonarSource S.A., Geneva, Switzerland, White Paper, 2018.

[6] SonarSource, "Java static code analysis rules," 2026. [Online]. Available: https://rules.sonarsource.com/java/

[7] Mountainminds GmbH & Co. KG, "JaCoCo Java Code Coverage Library," 2026. [Online]. Available: https://www.jacoco.org/jacoco/

[8] M. Fowler, "Catalog of Refactorings," 2026. [Online]. Available: https://refactoring.com/catalog/

[9] M. Fowler, K. Beck, J. Brant, W. Opdyke and D. Roberts, *Refactoring: Improving the Design of Existing Code*. Reading, MA, USA: Addison-Wesley, 1999.